

NOMBRES: Alejandro José

APELLIDOS: Gómez Hernández

CARNE: 20347

FECHA: 28 julio 2024

Enlace al repositorio: https://github.com/aleg001/S2_NLP_2024

Sucesión de Fibonacci

Para esta tarea se programará Fibonacci recursivo y con programación dinámica.

El primer approach es el de recursión, donde se llama a Fibonacci con los valores de `n-1` y `n-2` hasta llegar a los casos base.

El segundo approach es el de programación dinámica, que básicamente es almacenar los valores en un diccionario y evitar calcularlos nuevamente.

Así mismo, cabe destacar que se presentará la complejidad en notación O de ambos algoritmos.

1. Approach de Fibonacci de forma recursiva.

```
In [ ]: def fib_approach_recursivo(n, depth=0):
        """
        Calcula el n-ésimo número de Fibonacci de manera recursiva con información de depuración.

        Args:
            n (int): El índice del número Fibonacci.
            depth (int): Profundidad de recursión.

        Returns:
            int: N-ésimo Fibonacci.
        """
        indent = " " * depth
        print(f"{indent}fib_approach_recursivo({n})")

        if n <= 1:
            print(f"{indent}Devolviendo {n}")
            return n
        prelimA = fib_approach_recursivo(n - 1, depth + 1)
        prelimB = fib_approach_recursivo(n - 2, depth + 1)
        resultado = prelimA + prelimB
        print(f"{indent}Devolviendo {resultado}")
        return resultado
```

Nota aclaratoria y análisis:

Es fácil de implementar, sí. Sin embargo, no es eficiente ya que le toca calcular nuevamente muchos valores, lo que no lo hace tan sostenible o tan friendly con la memoria, en mi opinión.

Complejidad Temporal del approach recursivo:

La complejidad temporal de este algoritmo es de $O(2^n)$, ya que se llama a la función dos veces en cada llamada recursiva.

2. Approach de Fibonacci de forma dinámica.

```
In [ ]: def fib_approach_dinam(n):
        """
        Calcula el n-ésimo Fibonacci con progra dinámica.

        Args:
            n (int): El índice del número Fibonacci.

        Returns:
            int: N-ésimo Fibonacci.
        """
        if n <= 1:
            return n

        # Guardar subproblemas
        fib = [0] * (n + 1)
        fib[1] = 1

        print(f"Inicialización: {fib}")

        for i in range(2, n + 1):
            tempFib1 = fib[i - 1]
            tempFib2 = fib[i - 2]
            fib[i] = tempFib1 + tempFib2
            # Impresión
            print(f"fib[{i}]")
            print("=")
            print(f"fib[{i - 1}] + fib[{i - 2}]" )
            print(">")
            print(f"fib[{i}]")

        return fib[n]
```

Al hacerlo dinámico, tenemos **O(n)** ya que calculamos cada valor una sola vez. Esto debido a que guardamos todo, así que no calculamos otra vez. Vemos que es lineal, mucho más eficiente, y ayuda más en temas de poder de cómputo.

Probemos ambos para ver comportamiento.

```
In [ ]: print("Fibonacci Recursivo:")
        print(f"Recursivo con 10: {fib_approach_recursivo(10)}")
```

[illegible]

[illegible]

```

    Devolviendo 1
    Devolviendo 2
    Devolviendo 5
    fib_approach_recursivo(4)
    fib_approach_recursivo(3)
    fib_approach_recursivo(2)
    fib_approach_recursivo(1)
    Devolviendo 1
    fib_approach_recursivo(0)
    Devolviendo 0
    Devolviendo 1
    fib_approach_recursivo(1)
    Devolviendo 1
    Devolviendo 2
    fib_approach_recursivo(2)
    fib_approach_recursivo(1)
    Devolviendo 1
    fib_approach_recursivo(0)
    Devolviendo 0
    Devolviendo 1
    Devolviendo 3
    Devolviendo 8
    fib_approach_recursivo(5)
    fib_approach_recursivo(4)
    fib_approach_recursivo(3)
    fib_approach_recursivo(2)
    fib_approach_recursivo(1)
    Devolviendo 1
    fib_approach_recursivo(0)
    Devolviendo 0
    Devolviendo 1
    fib_approach_recursivo(1)
    Devolviendo 1
    Devolviendo 2
    fib_approach_recursivo(2)
    fib_approach_recursivo(1)
    Devolviendo 1
    fib_approach_recursivo(0)
    Devolviendo 0
    Devolviendo 1
    Devolviendo 3
    fib_approach_recursivo(3)
    fib_approach_recursivo(2)
    fib_approach_recursivo(1)
    Devolviendo 1
    fib_approach_recursivo(0)
    Devolviendo 0
    Devolviendo 1
    fib_approach_recursivo(1)
    Devolviendo 1
    Devolviendo 2
    Devolviendo 5
    Devolviendo 13
    fib_approach_recursivo(6)
    fib_approach_recursivo(5)
    fib_approach_recursivo(4)
    fib_approach_recursivo(3)
    fib_approach_recursivo(2)
    fib_approach_recursivo(1)
    Devolviendo 1
    fib_approach_recursivo(0)
    Devolviendo 0
    Devolviendo 1
    fib_approach_recursivo(1)
    Devolviendo 1
    Devolviendo 2
    fib_approach_recursivo(2)
    fib_approach_recursivo(1)
    Devolviendo 1
    fib_approach_recursivo(0)
    Devolviendo 0
    Devolviendo 1
    Devolviendo 3
    fib_approach_recursivo(3)
    fib_approach_recursivo(2)
    fib_approach_recursivo(1)
    Devolviendo 1
    fib_approach_recursivo(0)
    Devolviendo 0
    Devolviendo 1
    fib_approach_recursivo(1)
    Devolviendo 1
    Devolviendo 2
    Devolviendo 5
    fib_approach_recursivo(4)
    fib_approach_recursivo(3)
    fib_approach_recursivo(2)
    fib_approach_recursivo(1)
    Devolviendo 1
    fib_approach_recursivo(0)
    Devolviendo 0
    Devolviendo 1
    fib_approach_recursivo(1)
    Devolviendo 1
    Devolviendo 2
    fib_approach_recursivo(2)
    fib_approach_recursivo(1)
    Devolviendo 1
    fib_approach_recursivo(0)
    Devolviendo 0
    Devolviendo 1
    Devolviendo 3
    Devolviendo 8
    Devolviendo 21
    Devolviendo 55
    Recursivo con 10: 55

```

```

In [ ]: print("\nFibonacci Dinámico:")
        print(f"Recursivo con 10: {fib_approach_dinam(10)}")

```

```
Fibonacci Dinámico:
Inicialización: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
fib[2]
=
fib[1] + fib[0]
->
fib[i]
fib[3]
=
fib[2] + fib[1]
->
fib[i]
fib[4]
=
fib[3] + fib[2]
->
fib[i]
fib[5]
=
fib[4] + fib[3]
->
fib[i]
fib[6]
=
fib[5] + fib[4]
->
fib[i]
fib[7]
=
fib[6] + fib[5]
->
fib[i]
fib[8]
=
fib[7] + fib[6]
->
fib[i]
fib[9]
=
fib[8] + fib[7]
->
fib[i]
fib[10]
=
fib[9] + fib[8]
->
fib[i]
Recursivo con 10: 55
```